

ASR Shootout for Indian Conversational Speech

Benchmarking Deepgram, Whisper Large-v3, and Sarvam AI for Locality Recognition

Extended with HuggingFace Kambath (AI4Bharat) Hindi Dataset

Candidate: Sagnick Das

Role: AI Intern Assessment

1. Introduction

This project evaluates Automatic Speech Recognition (ASR) systems for Indian conversational speech in real-world conditions. The primary goal was not only to measure transcription quality, but also to evaluate whether these systems can reliably capture locality names from noisy, informal, multilingual speech.

This matters directly for voice-based hiring platforms where users interact over phone calls or WhatsApp voice notes. In such systems, ASR failure is rarely about full-sentence understanding. The actual business failure happens when critical entities such as locality names, job roles, or salary expectations are transcribed incorrectly.

The evaluation was originally performed using a custom dataset of 20 self-recorded Bangalore locality utterances under varied acoustic conditions. This extended version adds 20 Hindi speech samples from the **AI4Bharat Kambath dataset** (sourced via HuggingFace), bringing the total to 40 samples and testing pure Devanagari Hindi transcription in addition to Hinglish code-switched speech.

All models were benchmarked against Deepgram, which was used as the baseline as required in the assignment.

The three systems evaluated were:

Model	Type	Reason for Selection
Deepgram Nova-2	API-based commercial ASR	Baseline model with strong production reputation
Whisper Large-v3	Open-source multilingual ASR	State-of-the-art open-source benchmark model
Sarvam AI Saarika	Indic-focused ASR API	Optimized specifically for Indian languages and accents

2. Dataset and Evaluation Setup

2.1 Custom Conversational Dataset (20 samples)

A custom dataset was recorded consisting of 20 conversational utterances containing Bangalore locality names such as Koramangala, Whitefield, HSR Layout, Marathahalli, and Rajajinagar.

Instead of reading isolated words, the recordings were embedded naturally inside conversational sentences such as:

┆ *"Haan, main Koramangala mein rehta hoon."*

This was done intentionally because production traffic is conversational, not studio-clean keyword audio. The recordings were intentionally varied across multiple acoustic conditions:

Condition	Description
Clean	Quiet indoor recording
Street Noise	Traffic/background outdoor noise
Phone Call	Telephony-compressed audio
Whispered	Low-energy speech
Rushed	Fast conversational delivery

2.2 HuggingFace Kambath Extension (20 samples)

To evaluate model performance on formal Hindi speech (beyond Hinglish), 20 samples were loaded from the **AI4Bharat Kambath dataset** via HuggingFace (`ai4bharat/Kambath`). These samples consist of read/broadcast-style Hindi sentences in Devanagari script, providing a different challenge from the conversational locality utterances.

These samples are logged with `condition = "Hugging_face"` and `locality = "hf"` in the manifest. The LEA metrics are not meaningful for this subset (there are no Bangalore localities to detect), but WER and CER provide a direct measure of each model's Hindi transcription fidelity.

Total dataset: 40 samples across all conditions.

2.3 Metrics Used

Traditional Word Error Rate (WER) alone is insufficient for this task. For example:

"Main Koramangala mein rehta hoon"

"Main Kormangla mein rehta hoon"

may produce a relatively low WER penalty while still completely breaking downstream entity extraction. Because of this, the evaluation used multiple metrics:

Metric	Purpose
WER	Measures word-level transcription accuracy
CER	Measures character-level transcription accuracy
LEA Exact	Exact locality extraction accuracy
LEA Fuzzy	Fuzzy locality matching accuracy ($\geq 60\%$ char similarity)
Latency	Measures inference responsiveness
Cost	Important for production scalability

LEA (Locality Extraction Accuracy) remains the most business-relevant metric for the locality subset.

3. Benchmark Results (40 Samples)

3.1 Overall Performance

Model	WER ↓	CER ↓	LEA Exact ↑	LEA Fuzzy ↑	Avg Latency	Cost/min
Deepgram Nova-2	65.3%	57.2%	0%	2.5%	1,395 ms	\$0.0059
Whisper Large-v3	96.6%	45.3%	0%	0%	2,811 ms	Free
Sarvam AI Saarika	100%	100%	0%	0%	554 ms	\$0.004

The aggregate WER figures shifted substantially compared to the original 20-sample run. Deepgram improved from 108% to 65.3% overall WER because it transcribed Hindi samples (WER 22.7%) far better than it handled Hinglish locality utterances. Whisper's overall WER dropped from 181% to 96.6% for the same reason. Sarvam produced no output for the HuggingFace subset and remained at 100% WER overall.

3.2 Performance by Acoustic Condition

Model	Clean	Street Noise	Phone Call	Whispered	Rushed	HuggingFace (Hindi)
Deepgram Nova-2	100% WER	100% WER	100% WER	127.5% WER	116.7% WER	22.7% WER
Whisper Large-v3	187.1% WER	176.3% WER	178.9% WER	188.6% WER	178.3% WER	12.0% WER
Sarvam AI Saarika	100% WER	100% WER	100% WER	100% WER	100% WER	100% WER

3.3 Key Observations

Whisper Large-v3 on Hindi (Kambath)

Whisper achieved 12.0% WER on the Kambath Hindi samples — a dramatic contrast to its 170–190% WER on Hinglish locality utterances. The model's multilingual pretraining gives it strong formal Hindi. Its weakness is code-switched Hinglish with hyper-local proper nouns not seen in training data.

Deepgram Nova-2 on Hindi (Kambath)

Deepgram achieved 22.7% WER on the Kambath samples. This was better than expected for an API model not specifically optimized for Indic languages. A subset of samples returned empty transcripts (WER = 100%), suggesting the model occasionally silently failed on certain Hindi utterances rather than attempting transcription.

Sarvam AI Saarika on Hindi (Kambath)

Sarvam returned empty transcripts for all 20 HuggingFace samples (WER = 100%), which is unexpected given its stated Indic language support. This may reflect an API configuration mismatch (e.g., language code or endpoint not set for the Kambath audio format) rather than a fundamental model limitation.

Locality detection remains zero across all models

Neither the original 20 samples nor the Kambath extension produced any LEA Exact hits. The locality samples continue to fail due to hyper-local proper noun distortion (Byatarayanapura → "Bayatarayanpur", Kadugondanahalli → fragmented tokens). The Kambath samples have no locality targets, so they do not contribute to LEA metrics.

4. Failure Analysis

4.1 Locality Name Distortion

Rare locality names caused major issues across all systems on the original 20 samples. This is unchanged by the Kambath addition.

Actual	Sample Incorrect Predictions
Byatarayanapura	"Bayatarayanpur", "Byatranpur"
Kadugondanahalli	"Kadugonda halli", "Kadugondnahali"
Rajarajeshwarinagar	Fragmented into multiple tokens

These failures matter because downstream entity systems depend on correct locality recognition.

4.2 Noise Sensitivity

Street-noise recordings caused large degradation for all models on the locality subset. Whisper remained the most robust overall. The Kambath samples, being cleaner recordings, did not expose noise-related issues for those models that transcribed them.

4.3 Whispered Speech

Whispered recordings exposed a common weakness: low-energy speech caused models to hallucinate missing words or partially infer phrases from context. This becomes dangerous in production because the transcription may appear fluent while still being factually incorrect.

4.4 Code-Switching vs. Monolingual Hindi

The Kambath results reveal a clear split in model capability:

- **Whisper** handles monolingual Hindi well but struggles severely with Hinglish code-switching + local proper nouns.
- **Deepgram** shows moderate Hindi ability but also fails at Hinglish locality names.
- **Sarvam** did not produce output for the Hindi samples in this run despite its Indic language positioning.

This suggests the Hinglish + hyper-local-name combination is a distinct challenge that no current model solves reliably.

4.5 Sarvam Silent Failure

Sarvam's 100% WER on all Kambath samples (empty transcripts) warrants investigation.

Possible causes include language detection mismatch, API parameter configuration, or audio format incompatibility with that endpoint. A re-run with explicit `language=hi` and normalised sample rates is recommended before drawing conclusions about Sarvam's Hindi capability.

5. Production Considerations

ASR selection is not only an accuracy problem. In production systems, deployment constraints matter equally.

Factor	Deepgram	Whisper Large-v3	Sarvam AI
Setup Complexity	Low	High	Low
GPU Requirement	No	Yes (recommended)	No
Latency	Good (1,395 ms)	Slow (2,811 ms)	Excellent (554 ms)
Offline Support	No	Yes	Limited
Indic Speech Handling	Moderate	Excellent	Strong (when working)
Scaling Cost	Usage-based	Infrastructure-based	Usage-based
Hindi Transcription (Kambath)	22.7% WER	12.0% WER	Failed (this run)

Whisper provides the best raw transcription quality, particularly for Hindi, but introduces significant operational cost: large VRAM requirements, high inference latency, and heavy runtime dependencies. It is difficult to deploy on low-memory environments.

Deepgram is easier to scale operationally and showed better-than-expected Hindi performance, but continues to struggle with Hinglish locality names that are uncommon in western-centric training data.

Sarvam's latency advantage (554 ms mean, 1,023 ms P95) is compelling for real-time applications, but the API failure on Kambath samples needs to be resolved before it can be trusted for Hindi workloads.

6. Recommendation

For production deployment in an Indian voice hiring platform, the findings from both the original and extended benchmark point to the same hybrid strategy:

1. **Use Deepgram or Sarvam for fast real-time inference** on conversational Hinglish speech.
2. **Route low-confidence or pure Hindi samples to Whisper** for secondary processing (12% WER on Kambath).
3. **Add a post-ASR entity correction layer** using fuzzy matching for locality names — since LEA remains 0% across all models regardless of overall WER.

This hybrid approach balances accuracy, latency, infrastructure cost, and scalability.

A pure WER-focused evaluation would miss this conclusion entirely. The benchmark also showed that entity extraction accuracy matters more than perfect sentence transcription in telephony hiring workflows.

7. Limitations

This evaluation still has several limitations:

- Small custom dataset size (20 locality samples, 20 Kambath samples)
- Limited speaker diversity in the locality subset (single speaker)
- No streaming ASR benchmark
- Sarvam failure on Kambath samples not yet diagnosed — results may change with correct API configuration
- LEA metrics only meaningful for the locality subset, not Kambath
- No rigorous throughput testing under concurrent load
- Kambath samples represent broadcast Hindi, not conversational Hindi

Future work could include:

- Larger multi-speaker locality datasets
 - Streaming latency benchmarks
 - Domain-specific fine-tuning on Bangalore locality names
 - Post-ASR entity correction models
 - Evaluation across additional Indian languages (Kannada, Tamil, Telugu)
 - Re-running Sarvam with corrected API parameters for Hindi
-

8. Conclusion

This project demonstrated that evaluating ASR systems for Indian conversational speech requires more than comparing WER scores.

The extended benchmark with the HuggingFace Kambath dataset added an important dimension: models that fail catastrophically on Hinglish locality utterances can perform well on formal Hindi. Whisper Large-v3 achieved 12% WER on Kambath samples while simultaneously producing 170–190% WER on Hinglish locality audio — exposing that code-switching and hyper-local proper nouns are the true challenge, not Hindi speech itself.

The benchmark revealed:

- **Whisper Large-v3** delivers the best raw transcription quality, especially for monolingual Hindi
- **Deepgram** offers the best operational simplicity, good latency, and surprisingly competitive Hindi performance
- **Sarvam AI** provides the best latency but had API-level issues with the HuggingFace samples in this run
- **Locality extraction accuracy (LEA) remains 0%** across all models — post-ASR correction is non-optional for this use case
- **The Hinglish + hyper-local-name combination** is a distinct unsolved challenge for all tested systems

The most important insight from this benchmark is that deployment constraints, entity-level accuracy, and language-condition coverage matter just as much as aggregate benchmark scores. An ASR system that performs well on Kambath may still fail in production if it cannot reliably recognize conversational Indian locality names under noisy telephony conditions — and vice versa.